Ordenamiento por Intercambio (Bubble Sort), del juego de tarjetas al código

Objetivo

Comprender el algoritmo de **Bubble Sort** mediante la simulación con tarjetas. Después, diseñar una solución en **pseudocódigo** e implementarla en **Java**.

Materiales

- Tarjetas con números o nombres (mínimo 5). Ejemplo: 34, 12, 25, 8, 19.
- Hojas para registrar las pasadas.
- Computadora con JDK instalado y Bloc de Notas

Fase 1: Simulación con tarjetas (se conserva de la actividad original)

1. Formen equipos de 5 o más personas. Cada integrante es una tarjeta y se colocan en fila.
2. Comparen pares adyacentes de izquierda a derecha. Si la tarjeta de la izquierda es mayor que la de la derecha, intercambian lugares.
3. Recorrer toda la fila es **una pasada**. Repitan las pasadas hasta que ya no haya intercambios.
4. **Registren en una tabla** el estado de la fila al terminar cada pasada, el número de comparaciones y el número de intercambios.

Fase 2: Propuesta de solución en pseudocódigo (en binas)

A partir del registro de la Fase 1, escriban su propio pseudocódigo. Debe responder a estas preguntas:

1. **¿Qué estructura de datos representa la fila de tarjetas?**
   Un arreglo de elementos porque una fila de tarjetas se puede representar como un arreglo de elementos
2. **¿Qué ciclo representa las pasadas y cuál las comparaciones?**
   Un ciclo externo para las pasadas y un ciclo interno para las comparaciones
3. **¿Cómo se realiza el intercambio de dos elementos sin perder uno de ellos?**
   Usando una variable temporal.
4. **¿Cómo sabe el algoritmo que ya terminó?**
   Cuando hace la pasada, ya no hay intercambios y el numero mas grande ya esta hasta el final
5. **¿Cuántos elementos hay que comparar en cada pasada?**
   En cada pasada uno menos porque el elemento más grande ya quedó en su lugar que es hasta el final

Entregable: el pseudocódigo en hoja o archivo, con nombres de variables claros y sangría.

Fase 3: Implementación en Java

1. Traduzcan su pseudocódigo a un programa en Java con un método bubbleSort(int[] arreglo).
2. El programa debe mostrar el arreglo **después de cada pasada**, para compararlo con el registro de la Fase 1.
3. Debe contar e imprimir el **total de comparaciones, intercambios y pasadas**.
4. Prueben con el arreglo {34, 12, 25, 8, 19}, con uno ya ordenado y con uno en orden inverso.
5. Variación opcional: ordenar un arreglo de String alfabéticamente con compareTo.

Pseudocódigo

```
Definir arreglo, i, j, aux Como Entero
   Dimension arreglo[5]

   arreglo[1] <- 6
   arreglo[2] <- 7
   arreglo[3] <- 3
   arreglo[4] <- 10
   arreglo[5] <- 17

   Para i <- 1 Hasta 4 Hacer
      Para j <- 1 Hasta 5 - i Hacer
         Si arreglo[j] > arreglo[j + 1] Entonces
            aux <- arreglo[j]
            arreglo[j] <- arreglo[j + 1]
            arreglo[j + 1] <- aux
         FinSi
      FinPara

      Escribir "Pasada ", i, ": "
      Para j <- 1 Hasta 5 Hacer
         Escribir Sin Saltar arreglo[j], " "
      FinPara

      Escribir ""

   FinPara

FinProceso
```

Fase 4: Reflexión

1. ¿Coincide la salida con las tarjetas?
Sí el programa hace los mismos intercambios que hicimos a mano y al final el arreglo queda ordenado 3 6 7 10 17.

2. ¿Qué cambios tuvo el pseudocódigo al pasarlo a Java?

Los índices empiezan en 0 y no en 1.

Escribir pasó a ser System.out.print.

Para pasó a ser for, y Si pasó a ser if.

3. ¿Qué pasa si el arreglo ya está ordenado? ¿Sigue comparando?
Sí, sigue comparando todo, aunque ya no haya nada que cambiar.

4. ¿Es eficiente con listas muy grandes?
No compara cada número con casi todos los demás así que con listas grandes tarda mucho. Eso es $O(n^2)$ si la lista crece 10 veces tarda 100 veces más.

Lista de cotejo (5 puntos)

#	Criterio	Cumple	No cumple	Observaciones
1	**Simulación y registro:** la tabla de pasadas con tarjetas es completa y correcta (estado de la fila, comparaciones e intercambios por pasada).			
2	**Pseudocódigo:** es claro, está estructurado y contiene los dos ciclos (pasadas y comparaciones), la condición de comparación, el intercambio con variable auxiliar y la condición de paro.			
3	**Implementación en Java:** el programa compila, ejecuta y ordena correctamente el arreglo de ejemplo y los casos de prueba (ordenado e inverso).			
4	**Coherencia pseudocódigo–código:** el programa respeta la lógica del pseudocódigo propuesto y muestra el arreglo tras cada pasada, junto con los contadores.			
5	**Reflexión y buenas prácticas:** responde las preguntas de reflexión con argumentos, incluida la eficiencia, y el código usa nombres significativos, sangría y comentarios.			